SMART PARKING SLOT ALLOCATOR

Topic: Smart Parking Slot Allocator

PROJECT REPORT

A Web-Based Solution for Efficient Parking Slot Management

Submitted By
TEAM 3

Team Members
C. Akshitha - 294
N. Sadaya Sri - 333
CH. Kavya - 296
R. Neha - 348

Department
CSE - AIML

College
Malla Reddy Vishwavidyapeeth Technical Campus

Academic Year
2026–2027

[COLLEGE LOGO / PROJECT IMAGE]

This report presents the design, implementation, and testing of a web-based parking management application.

1. ABSTRACT AND INTRODUCTION

1.1 Abstract

The Smart Parking Management System is a web-based application designed to make parking slot management simpler, faster, and more organized. The system provides a user interface through which a person can log in, enter or select vehicle information, view the parking map, and identify parking slots. The application represents slot status so that users and administrators can understand which spaces are available and which are occupied. An administrator section is also included to manage parking information and monitor the overall parking area.

The project follows a simple full-stack structure. The frontend provides the user interface, JavaScript handles interaction and dynamic updates, and the backend/API layer can be used to connect the application with persistent data. The system is designed as an extendable foundation for features such as online booking, QR-based entry, notifications, payment, and automated vehicle identification.

1.2 Introduction

Finding and managing parking spaces can become difficult when the number of vehicles is high and information about available spaces is not clearly visible. Traditional parking management often depends on manual checking, which can consume time and create confusion for both users and parking administrators.

The proposed system addresses this issue by providing a digital parking interface. Instead of manually checking every space, users can view the parking map and understand the current status of slots. The application also separates user functionality from administrative functionality, making it easier to manage parking information.

1.3 Need for the System

- To provide a clear digital view of parking slots.
- To reduce manual effort in checking parking availability.
- To organize vehicle and slot information.
- To provide an administrator interface for parking management.
- To create a base that can later support smart parking features.

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

2.1 Problem Statement

In many parking areas, users do not have a convenient way to know which parking spaces are available before searching for a slot. Manual monitoring can lead to unnecessary movement inside the parking area, wasted time, and difficulty in maintaining accurate occupancy information. Administrators may also need a separate method to monitor and update parking status.

Therefore, there is a need for a simple digital system that displays parking slots, allows users to interact with the parking interface, and provides administrative control over parking information.

2.2 Objectives

- Develop a user-friendly web application for parking management.
- Provide secure-style login and navigation for users.
- Display parking slots through an interactive parking map.
- Represent occupied and available slot status clearly.
- Allow the user to enter vehicle information before accessing parking functionality.
- Provide an admin dashboard for monitoring and managing parking information.
- Create a modular architecture that can be extended with database and smart parking features.

2.3 Scope of the Project

The current project focuses on the core web application and parking management workflow. The main scope includes user login, dashboard navigation, vehicle number input, parking map display, slot status representation, and an admin dashboard. The project can later be expanded to include real-time sensor integration, online payments, QR codes, reservation management, notifications, and automated vehicle recognition.

2.4 Main Users

The system has two main categories of users:

- User: Uses the application to access the dashboard, enter vehicle information, view parking slots, and understand slot availability.
- Administrator: Uses the admin interface to monitor and manage parking-related information.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 System Architecture

The application follows a simple layered architecture. The frontend is responsible for displaying pages and collecting user actions. JavaScript controls page behavior, parking-slot interaction, and local application state. The backend/API layer provides a path for processing requests and connecting the frontend to a database when persistent storage is enabled.

USER
Login / Vehicle Number / Parking Interaction
FRONTEND
HTML + CSS + JavaScript
APPLICATION LOGIC
Dashboard + Slot Status + Booking/Selection Logic
BACKEND / API
Request Processing and Data Management
DATABASE
Users + Parking Slots + Booking Information

3.2 Project Modules

Module	Purpose	Main Function
Login	User access	Accepts login details and opens the application.
Dashboard	Main user area	Provides access to parking features.
Parking Map	Slot visualization	Displays parking slots and their status.
Vehicle Details	Vehicle identification	Stores/uses the entered vehicle number for the session.
Admin Dashboard	Management	Allows an administrator to monitor and manage parking information.

3.3 Basic Data Flow

The basic flow is: User → Login → Dashboard → Vehicle Number → Parking Map → Slot Status/Selection. For administration, the flow is: Administrator → Admin Login/Access → Admin Dashboard → Parking Information Management.

4. IMPLEMENTATION AND FEATURES

4.1 User Login

The login page provides an entry point to the application. It is designed to accept user credentials and redirect the user to the dashboard after successful login. A signup option can be provided for creating a user account. In a production version, authentication should be connected to a backend database and passwords should be securely hashed.

4.2 User Dashboard

After login, the user reaches the dashboard. The dashboard acts as the central navigation area for the parking system. It provides access to parking-related functions and presents information in a simple layout.

[INSERT SCREENSHOT: USER LOGIN PAGE]

[INSERT SCREENSHOT: USER DASHBOARD]

4.3 Vehicle Number Entry

Before viewing the parking slots, the user can enter the vehicle number. The application validates that the field is not empty and stores the vehicle number for use during the parking session. This helps associate the parking interaction with a specific vehicle.

4.4 Parking Map

The parking map is the main functional part of the application. It visually represents the parking area and its slots. Each slot can be represented according to its current state, such as available or occupied. This gives users a quick understanding of the parking situation.

[INSERT SCREENSHOT: PARKING MAP WITH AVAILABLE/OCCUPIED SLOTS]

4.5 Admin Dashboard

The admin dashboard is designed for parking management. It can display parking information and provide controls for maintaining slot status. Separating the admin area from the user dashboard makes the system easier to manage and extend.

[INSERT SCREENSHOT: ADMIN DASHBOARD]

5. TECHNOLOGIES, DATABASE AND TESTING

5.1 Technologies Used

Technology	Use	Purpose in Project
HTML	Frontend	Creates application pages and structure.
CSS	Frontend	Provides layout, styling, and visual design.
JavaScript	Frontend Logic	Handles user actions, navigation, and parking-slot interaction.
Node.js / Express	Backend	Provides server-side/API functionality.
Database	Data Layer	Stores user, slot, and parking-related information when connected.

5.2 Database Design

A database-based implementation can organize the application information into separate tables. A Users table can store account information, a ParkingSlots table can store slot identifiers and status, and a Bookings table can store vehicle and parking transaction details.

Table	Important Fields	Purpose
Users	user_id, email, password	Stores registered user information.
ParkingSlots	slot_id, slot_number, status	Stores parking slot information.
Bookings	booking_id, vehicle_no, slot_id, time	Stores parking/selection records.

5.3 Testing

The application should be tested module by module to verify that user actions produce the expected results.

Test Case	Expected Result	Status
Valid login	User reaches dashboard	Pass
Empty vehicle number	Validation message appears	Pass
Open parking map	Parking slots are displayed	Pass
Occupied slot interaction	Occupied slot cannot be selected	Pass
Open admin page	Admin interface is displayed	Pass

6. RESULTS, FUTURE SCOPE AND CONCLUSION

6.1 Results

The developed prototype provides the main workflow required for a digital parking management application. Users can access the system, move to the dashboard, provide vehicle information, and view parking slots through a parking map. The application also includes a separate administrative interface for managing parking information. The modular structure makes it possible to add more advanced functionality without redesigning the complete application.

6.2 Advantages

- Simple and user-friendly interface.
- Clear visualization of parking slot status.
- Reduces the need for manual slot checking.
- Separate user and administrator functionality.
- Can be connected to a database for persistent parking records.
- Can be extended to support smart parking technologies.

6.3 Future Scope

- Real-time parking sensors for automatic slot-status updates.
- Online slot reservation and cancellation.
- QR-code based entry and exit.
- Online payment integration.
- SMS/email/push notifications.
- Automatic number-plate recognition.
- Mobile application for Android/iOS.
- Analytics dashboard showing occupancy and parking usage.

6.4 Conclusion

The Smart Parking Management System demonstrates how a web application can be used to organize parking operations digitally. The project focuses on the essential workflow of user access, dashboard navigation, vehicle identification, parking-slot visualization, and administrative management. It provides a practical foundation that can be improved with real-time data, databases, sensors, payment systems, and intelligent automation in future versions.

6.5 References

- HTML, CSS and JavaScript web development documentation.
- Node.js and Express documentation.
- Database management concepts and SQL documentation.
- Project source code and testing performed during development.

— END OF PROJECT REPORT —